BIOMEDICAL IMAGE ANALYSIS • RESEARCH PRESENTATION

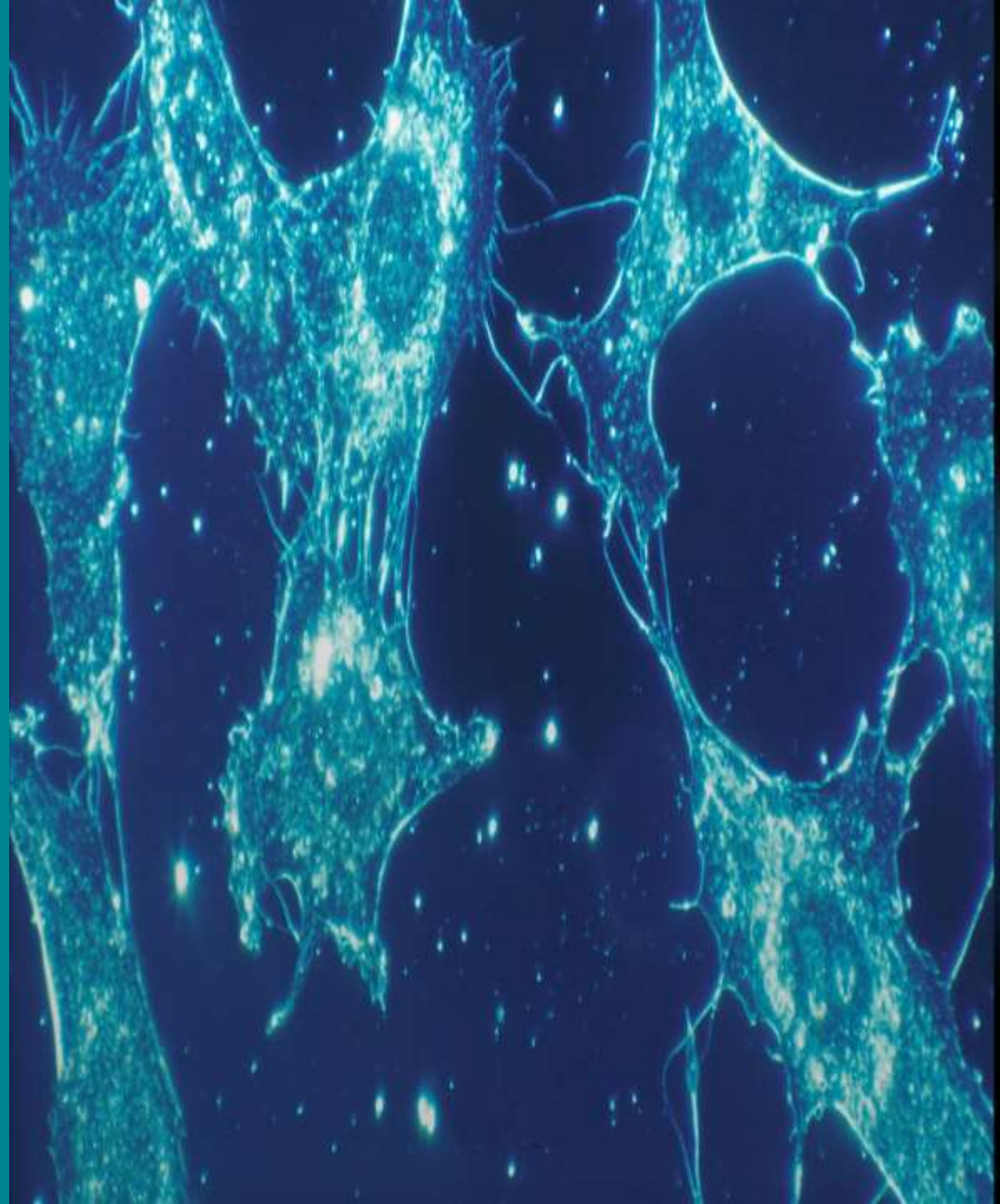
Malaria Detection from Cell Images

Traditional machine learning using HOG + LBP texture features

Dhvan Shah • Khush Chauhan

B.Tech (ENTC) | Machine Learning Project

Dataset: Cell Images for Detecting Malaria



Study workflow: from microscope image to class prediction

One consistent pipeline turns each cell image into a compact feature vector, then compares four modelling approaches.

01 — IMAGE DATA
27,558 cell images
13,779 per class

→

02 — PREPROCESS
64 × 64 grayscale
CLAHE • Smoothening • mask

→

03 — EXTRACT
HOG + LBP
1,774 descriptors

→

04 — SELECT
SelectKBest
retain 300 features

→

05 — MODEL
RF • KNN • K-means
Logistic Regression

→

06 — EVALUATE
Confusion matrix
accuracy • F1 • speed

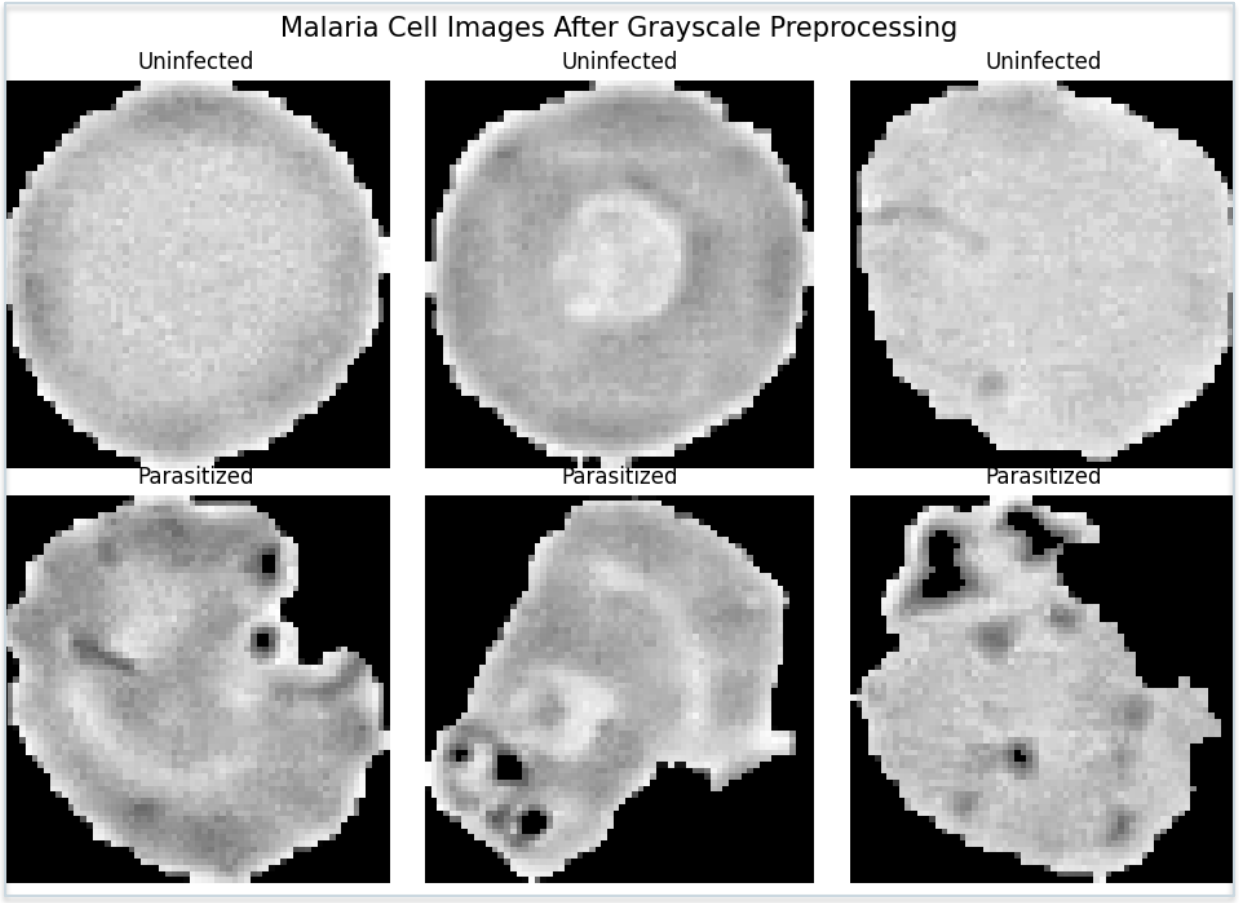
RESEARCH QUESTION

How accurately and efficiently can handcrafted cell-texture features distinguish parasitized from uninfected images?

Evaluation uses a held-out stratified test split; all four approaches are compared on the same binary classification task.

Dataset and target labels

The model predicts the label of a cell image; this is not a direct diagnosis of a patient.



Notebook figure: representative cell images / preprocessing examples

27,558 TOTAL IMAGES

Kaggle: Cell Images for Detecting Malaria

13,779

UNINFECTED | CLASS 0

Negative class

13,779

PARASITIZED | CLASS 1

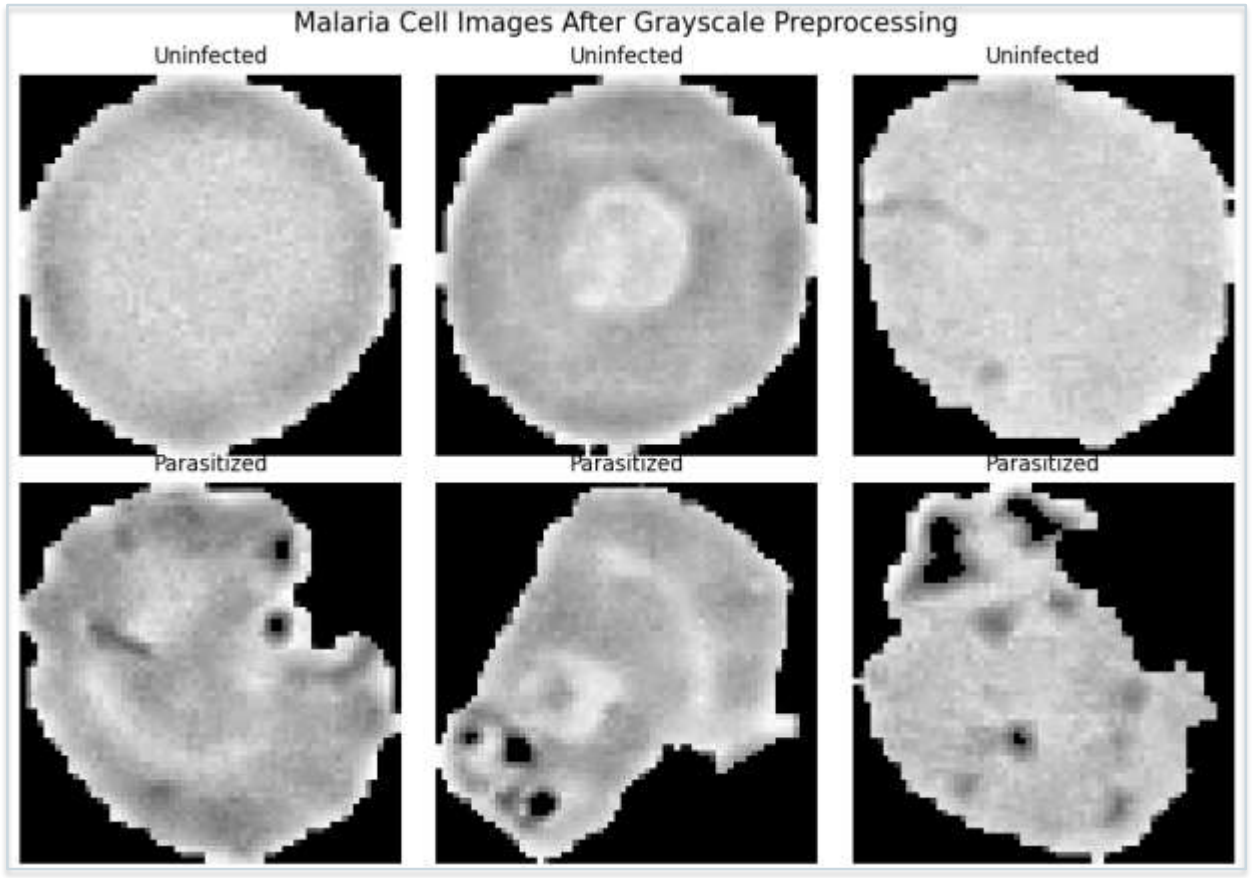
Positive class

WHY THE BALANCED DATASET HELPS

- Equal class counts make overall accuracy easier to interpret.
- Recall and the confusion matrix are still needed to see missed infections.

Preprocessing: standardize images before extracting texture

Every image passes through the same steps so differences in size, contrast and background are less likely to dominate.



Actual six-panel output from the notebook

1 Resize — 64 × 64 pixels
Creates a consistent input size.

2 Grayscale — Intensity image
Focuses the pipeline on texture and edges.

3 CLAHE — Local contrast enhancement
Makes local structures easier to distinguish.

4 Gaussian blur — Noise smoothing
Reduces small fluctuations before thresholding.

5 Otsu threshold + mask — Foreground separation
Helps isolate the cell and darken background regions.

OUTPUT → cleaned grayscale cell image for HOG + LBP

Gaussian Blur – Noise Smoothing

What it does

Gaussian Blur replaces each pixel with a weighted average of its neighbours. Weights follow a 2-D Gaussian (bell-shaped) distribution, so nearby pixels contribute more than distant ones.

Mathematical Formula

$$G(x, y) = (1 / 2\pi\sigma^2) \cdot e^{-(x^2 + y^2) / 2\sigma^2}$$

σ (sigma) controls the blur strength. Larger σ → stronger smoothing.

Why we use it

- Reduces high-frequency noise
- Prevents small intensity spikes from affecting Otsu thresholding
- Makes the subsequent cell-mask more stable

Conceptual Diagram

3×3 Gaussian Kernel (example)

1	2	1
2	4	2
1	2	1

Centre pixel gets the highest weight.
Surrounding pixels get lower weights.
Result = weighted average → smoothed image.

Reference: Gonzalez & Woods – Digital Image Processing (classic Gaussian filtering formulation)
https://homepages.inf.ed.ac.uk/rbf/HIPR2/gsmooth.htm

ANOVA F-test (f_classif) – Ranking Features

Purpose in our project

SelectKBest uses the ANOVA F-value to score every one of the 1,774 HOG+LBP features. Only the top 300 features that best separate Parasitized from Uninfected cells are kept.

F-statistic Formula

F = (Between-class variance) / (Within-class variance)

High F → feature means differ strongly between the two classes → more useful for classification.
Low F → feature values overlap a lot → less useful.

Important safeguard

Feature ranking is computed only on the training set. The selected 300 features are then applied to the test set. This prevents data leakage.

Selection Pipeline

1. Take the 1,774-dimensional training vectors
2. For each feature compute ANOVA F-value vs class label
3. Rank features from highest F to lowest F
4. Keep the top 300 features
5. Transform both train & test sets to 300-D

Reference: Scikit-learn documentation – sklearn.feature_selection.f_classif (ANOVA F-value)
https://www.geeksforgeeks.org/machine-learning/feature-selection-using-f-anova/

Otsu Threshold – Automatic Foreground Separation

What it does

Otsu's method finds the intensity threshold T that best separates the image into two classes (cell vs background) by maximising the between-class variance.

Core Formula

$$\sigma^2_B(T) = w_0(T) \cdot w_1(T) \cdot (\mu_0(T) - \mu_1(T))^2$$

w_0, w_1 = class probabilities
μ_0, μ_1 = class mean intensities
The T that maximises σ^2_B is chosen as the threshold.

Role in our pipeline

Creates a binary mask → isolates the cell → background is darkened → cleaner input for HOG + LBP.

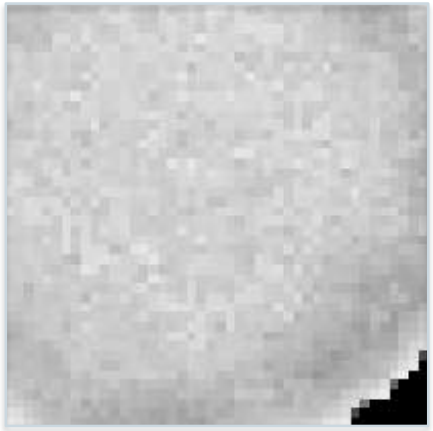
How the threshold is chosen

1. Build intensity histogram of the image
2. Try every possible threshold T
3. Compute between-class variance $\sigma^2_B(T)$
4. Select T with maximum σ^2_B
5. Apply T → binary mask (cell vs background)

Reference: Nobuyuki Otsu (1979) – A Threshold Selection Method from Gray-Level Histograms
https://engineering.purdue.edu/kak/computervision/ECE661.08/OTSU_paper.pdf

Feature extraction: represent edges and micro-texture numerically

HOG and LBP complement one another: one describes gradient structure; the other describes local intensity patterns.



Preprocessed cell

HOG | EDGE / SHAPE CUES

Histogram of Oriented Gradients

Measures gradient direction and strength in small image regions. It captures boundaries, contours and directional structure.

LBP | LOCAL TEXTURE CUES

Local Binary Patterns

Compares each pixel with its neighbours and forms a texture code. It captures fine local patterns that may differ across cell classes.

HOW THE DESCRIPTORS BECOME MODEL INPUT

HOG

324 gradient descriptors

→

LBP

1,450 local-texture descriptors

→

COMBINE

1,774 values in one vector

WHY THIS APPROACH? It converts an image into measurable features that classical machine-learning algorithms can compare without training a deep neural network.

HOG & LBP – Formulas and Diagram Descriptions

HOG – Histogram of Oriented Gradients

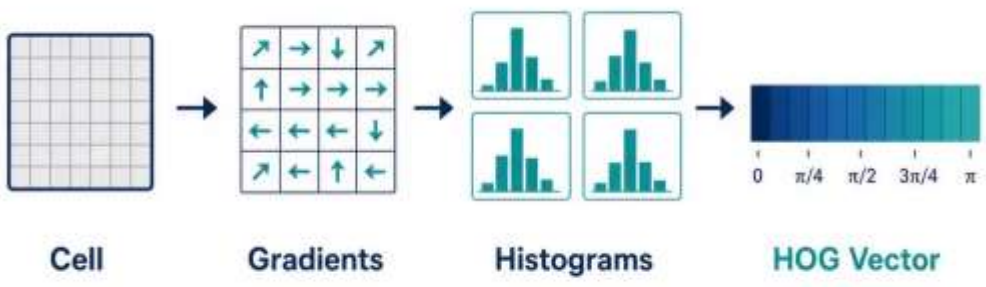
Step-by-step mathematics

1. Gradient at each pixel:
   $G_x = I(x+1,y) - I(x-1,y)$
   $G_y = I(x,y+1) - I(x,y-1)$

2. Magnitude & Orientation:
   $Mag = \sqrt{(G_x^2 + G_y^2)}$
   $\theta = \arctan(G_y / G_x)$

3. Image divided into cells → orientation histogram
4. Block normalisation → final descriptor

Pictorial description



LBP – Local Binary Patterns

Mathematical definition

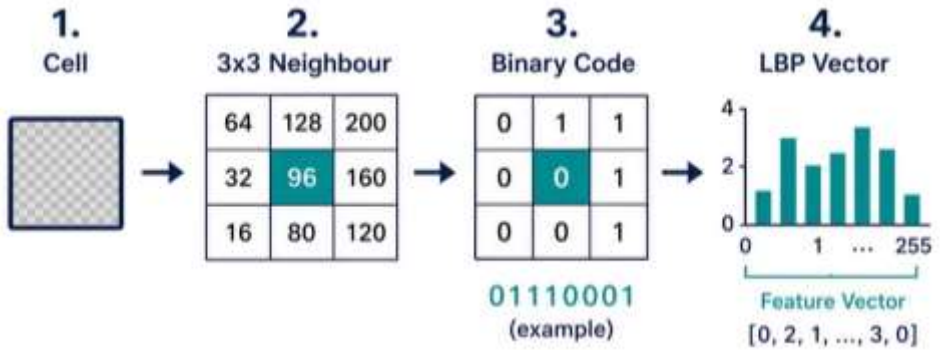
For centre pixel gc and P neighbours gp at radius R:

$LBP(P,R) = \Sigma\ s(gp - gc) \cdot 2^p \quad (p = 0 \ldots P-1)$

where $s(x) = 1$ if $x \geq 0$, else 0

Each pixel is turned into a binary code that encodes local texture. A histogram of these codes forms the feature vector.

Pictorial description



Feature selection: keep informative descriptors, discard the rest

SelectKBest with f_classif ranks each feature using an ANOVA F-statistic computed against the class labels.

1,774
Original HOG + LBP features

→

300
Features retained

83.1%
Reduction in dimensionality

SELECTION PROCEDURE

1 Fit on training data only

Avoids leaking test-set information into feature selection.

2 Score every descriptor

ANOVA F-value estimates how strongly a feature relates to the class.

3 Keep the top 300

The reduced vector is then used to train and evaluate the models.

Expected benefit: less computation and memory use. The experiment must still verify that the smaller feature set preserves predictive performance.

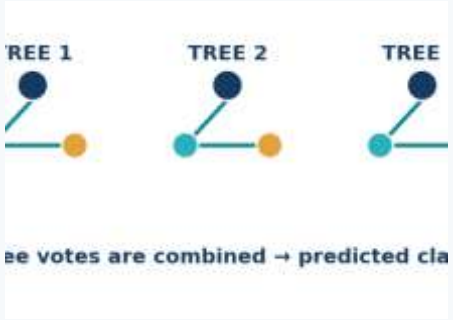
Four machine-learning approaches, four different decision strategies

Each approach receives the selected 300-feature vector; K-means is unsupervised and requires cluster-to-label matching for evaluation.

RANDOM FOREST

SUPERVISED • ENSEMBLE

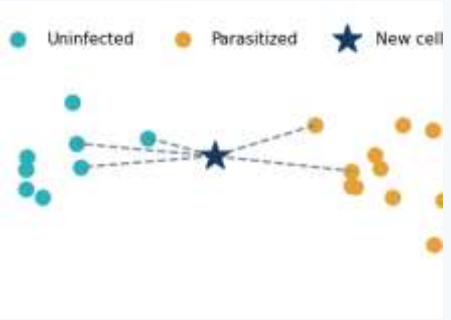
Builds many decision trees on varied samples/features, then combines their votes. Captures non-linear feature interactions.



K-NEAREST NEIGHBORS

SUPERVISED • DISTANCE

Finds the closest training examples in feature space and predicts from their labels. Sensitive to scaling and the choice of k.



K-MEANS

UNSUPERVISED • CLUSTERING

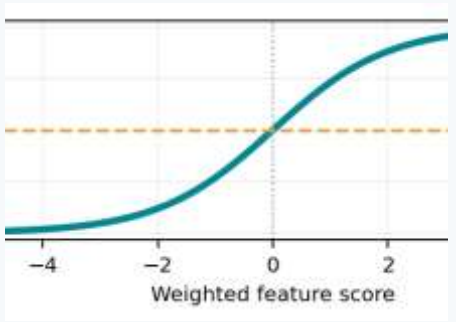
Groups feature vectors around centroids without using class labels during fitting. Hungarian matching aligns clusters to labels only for scoring.



LOGISTIC REGRESSION

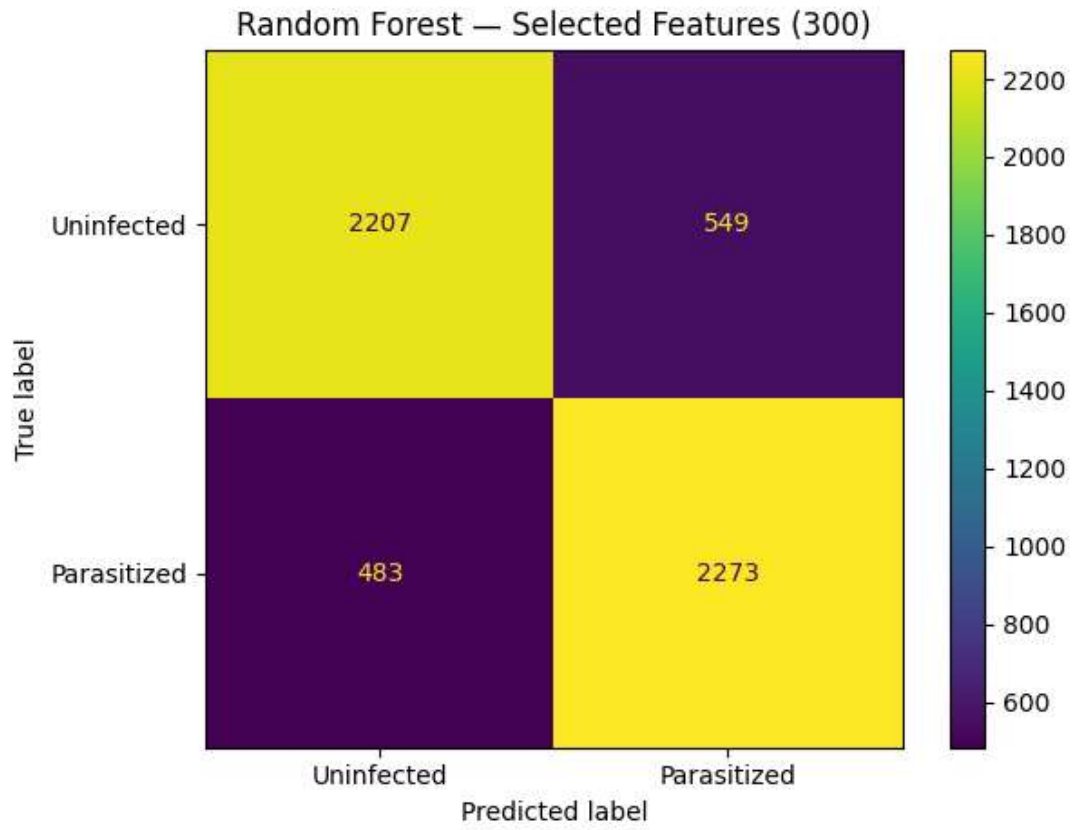
SUPERVISED • LINEAR CLASSIFIER

Combines weighted features and maps the score to a probability using a sigmoid function; a threshold gives the predicted class.



Fair comparison: same dataset split and selected features; compare accuracy and F1 alongside recall, specificity and inference time.

Random Forest: strong detection with balanced errors



Actual confusion matrix from the Python notebook

ACCURACY | 81.28%

Correct predictions across both classes.

RECALL | 82.48%

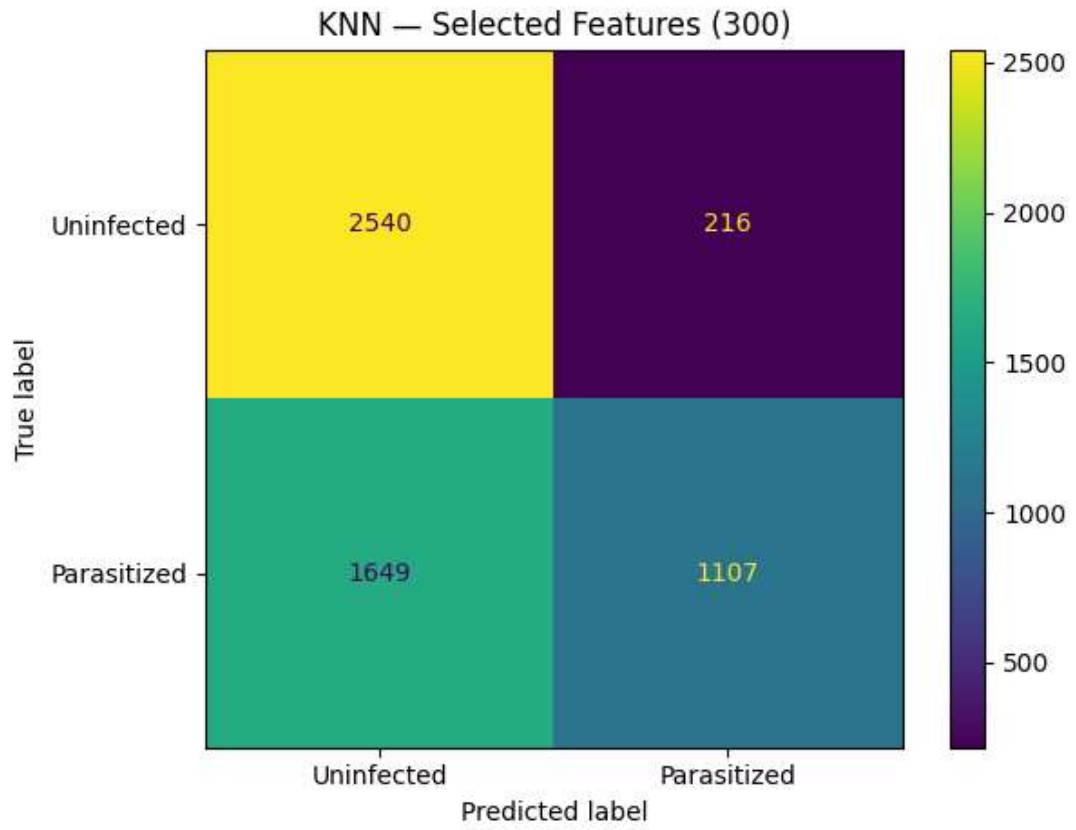
Detected most parasitized images in the test set.

F1 SCORE | 81.50%

Balances precision and recall.

Confusion matrix: TN 2,207 | FP 549
FN 483 | TP 2,273

K-nearest neighbors: conservative positive detection



Actual confusion matrix from the Python notebook

ACCURACY | 66.17%

Overall correctness is lower than the leading models.

SPECIFICITY | 92.16%

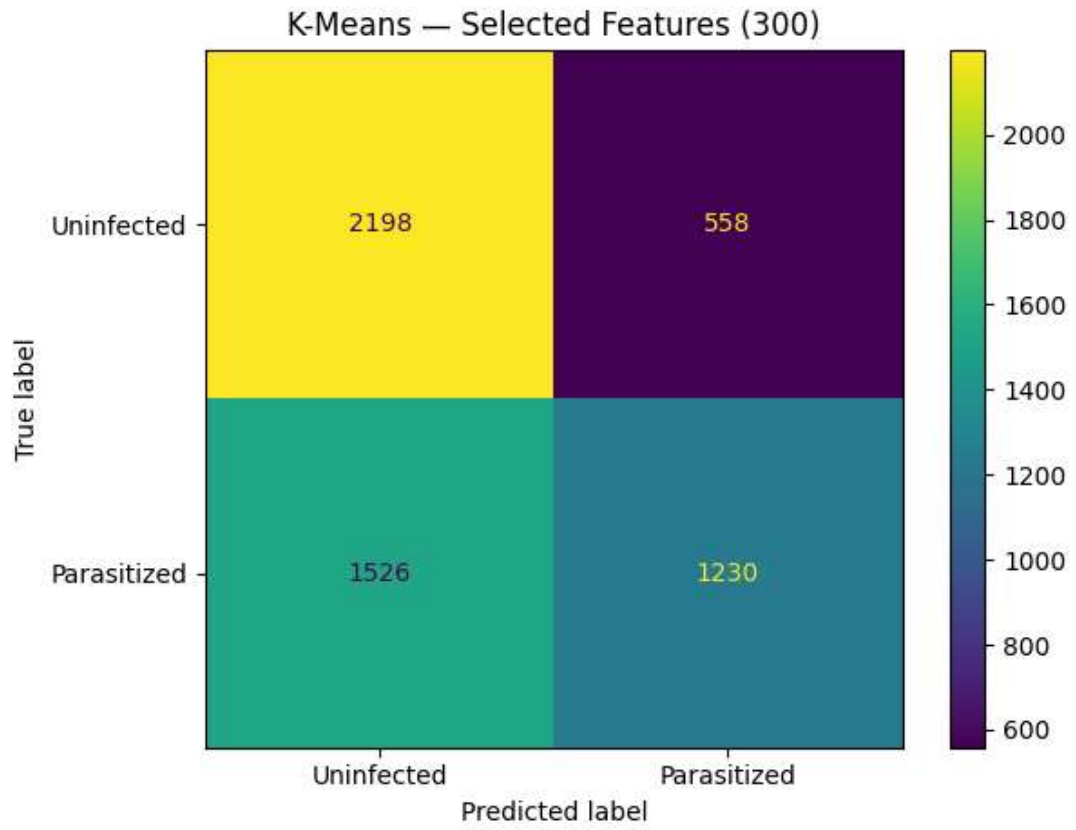
Most uninfected cells were correctly rejected as negative.

RECALL | 40.17%

Many parasitized cells were missed.

Confusion matrix: TN 2,540 | FP 216
FN 1,649 | TP 1,107

K-means: clustering is not naturally a classifier



Actual confusion matrix from the Python notebook

ACCURACY | 62.19%

Lowest overall accuracy among the four tested approaches.

PRECISION | 68.79%

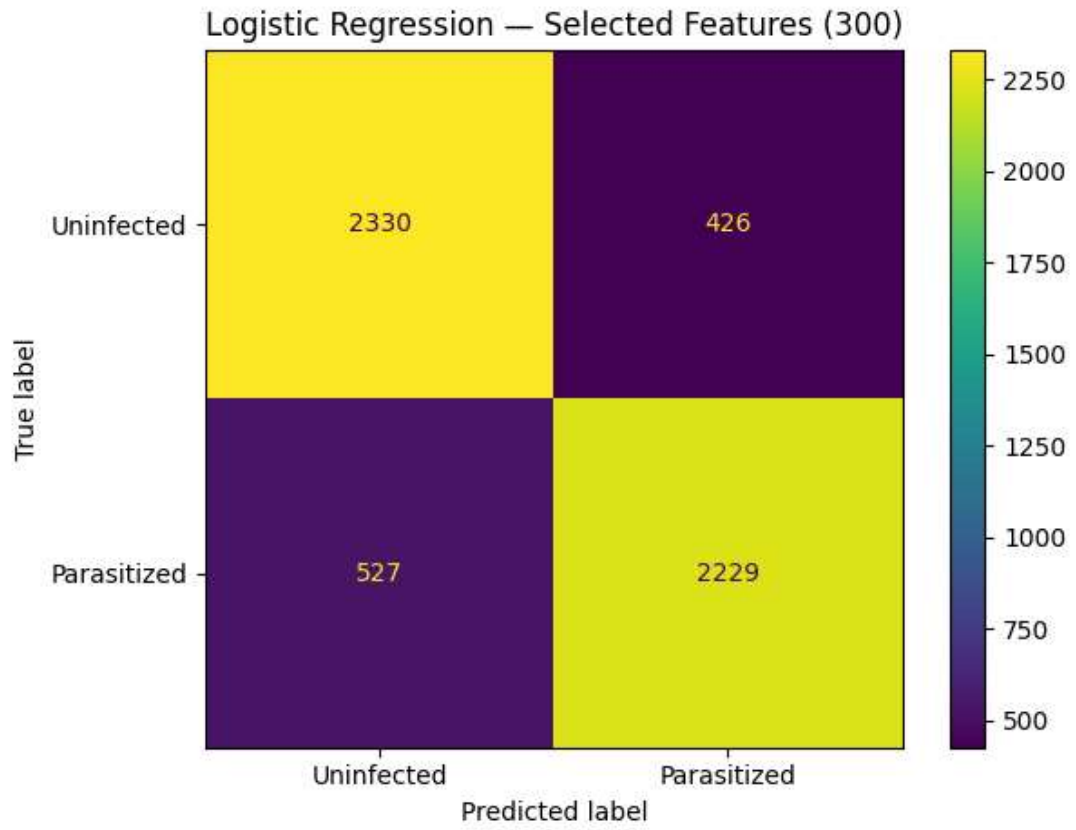
Predicted positive labels were correct about two-thirds of the time.

RECALL | 44.63%

Clustering missed many parasitized cells.

Confusion matrix: TN 2,198 | FP 558
FN 1,526 | TP 1,230

Logistic regression: best overall result in this comparison



Actual confusion matrix from the Python notebook

ACCURACY | 82.71%

Highest accuracy among the four models shown.

RECALL | 80.88%

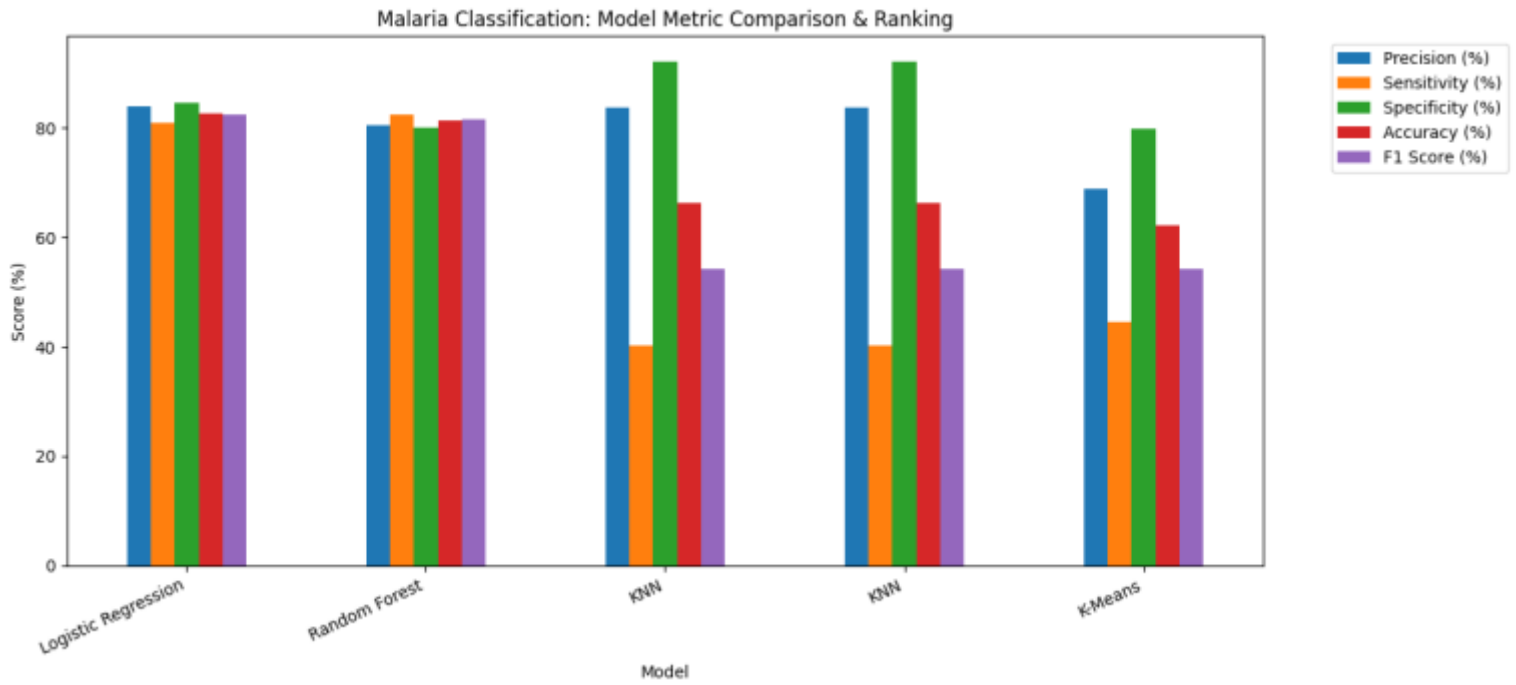
Detects a much larger share of parasitized cells than KNN or K-means.

F1 SCORE | 82.39%

Best balance of precision and recall in this experiment.

Confusion matrix: TN 2,330 | FP 426
FN 527 | TP 2,229

Model comparison: quality and speed together



Metric comparison chart generated by the submitted notebook

1 LOGISTIC REGRESSION

F1 82.39% | Accuracy 82.71%

2 RANDOM FOREST

F1 81.50% | Accuracy 81.28%

3–4 KNN / K-MEANS

F1 ≈54% | Lower recall

Logistic Regression is the strongest overall performer on the reported test split; Random Forest is a close second.

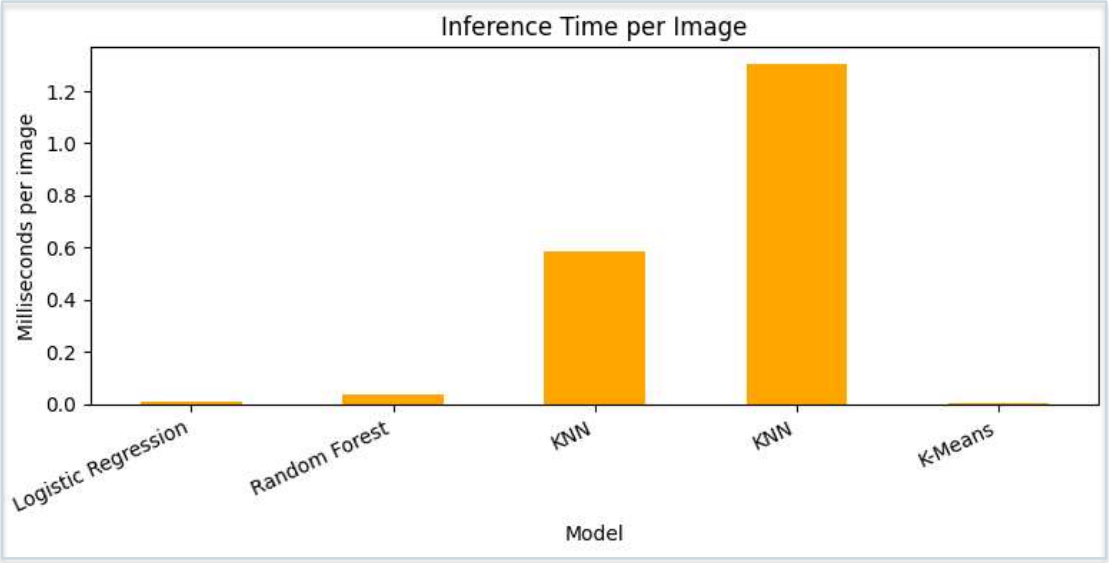
Efficiency and sensitivity to the train–test split

The experiment varies how much data is used for training while keeping Logistic Regression as the model under study.

TRAIN–TEST SPLIT EXPERIMENT

Train / Test	Train images	Test images	Accuracy	F1 score
80% / 20%	22,046	5,512	82.71%	82.39%
70% / 30%	19,290	8,268	82.20%	81.79%
60% / 40%	16,534	11,024	81.50%	81.00%
50% / 50%	13,779	13,779	81.23%	80.83%

INFERENCE TIME PER IMAGE



WHAT THE RESULTS SHOW

Accuracy falls from 82.71% to 81.23% as the training set is reduced from 80% to 50%. F1 follows the same modest downward trend.

SPEED RESULT

Logistic Regression inference: 0.010 ms/image in the reported run. Timing is hardware- and implementation-dependent.

Limitations and what must be validated next

The reported numbers describe this dataset and experimental setup; they should not be interpreted as clinical performance.

LIMITATIONS OF THE CURRENT STUDY

01	**Handcrafted descriptors only**	HOG + LBP were evaluated; no CNN or transfer-learning baseline was included.
02	**Low image resolution**	Resizing to 64 × 64 may remove fine cellular detail that could be diagnostically relevant.
03	**Limited validation**	The main results use one stratified train–test split; k-fold cross-validation and an independent test set were not reported.
04	**Model-specific errors**	KNN and K-means have low recall, so many parasitized images are missed in this experiment.
05	**Implementation caveat**	Notebook feature-selection comparison tables appeared to repeat values; those comparisons need checking before making claims about the benefit of selection.

NEXT VALIDATION STEPS

1. **Cross-validation**
   Use stratified k-fold splits to estimate variability, not just one test result.
2. **Independent images**
   Test on images from a separate source or laboratory to assess generalization.
3. **Benchmark deep learning**
   Compare CNN / transfer-learning baselines on the same train–test protocol.
4. **Clinical safeguards**
   Review false negatives carefully; do not use as a standalone diagnostic tool.

KEY POINT: promising baseline ≠ clinical validation

Final comparison and research takeaway

Logistic Regression is the strongest overall model in the reported experiment; Random Forest is a close second.

MODEL COMPARISON | SELECTED 300-FEATURE REPRESENTATION

Rank	Model	Accuracy	Precision	Recall	Specificity	F1 score	Inference / image
1	**Logistic Regression**	**82.71%**	**83.96%**	**80.88%**	**84.54%**	**82.39%**	**0.010 ms**
2	**Random Forest**	81.28%	80.55%	82.48%	80.08%	81.50%	0.035 ms
3	**KNN**	66.17%	83.67%	40.17%	92.16%	54.28%	1.303 ms
4	**K-Means**	62.19%	68.79%	44.63%	79.75%	54.14%	0.002 ms

WHAT WE LEARNED

- HOG + LBP encode useful cell texture in a compact vector.
- Reducing 1,774 features to 300 lowers input size substantially.
- Model choice matters: KNN/K-means miss more parasitized cases.

NEXT STEPS

- Verify feature-selection comparisons in the notebook.
- Run stratified k-fold and independent-source evaluation.
- Compare with CNN / transfer learning before stronger claims.

TAKE-HOME MESSAGE | A fast classical ML baseline is useful for comparison, but it is not a clinically validated malaria diagnostic system.